

---

# LAB MANUAL

## Introduction to Database Systems (SE2313)

**Prepared by**

Ms. Anam Zahra | Ms. Syeda Samar Fatima

**Reviewed by**

Ms. Itrat Fatima

**Summer 2025**

BS Software Engineering

**Department of Software Engineering**

Capital University of Science and Technology, Islamabad

---

---

# Contents

---

|          |                                                                            |           |
|----------|----------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction to SQL Tools and Techniques</b>                            | <b>1</b>  |
| 1.1      | Reference Material . . . . .                                               | 1         |
| 1.2      | Activity Timeline . . . . .                                                | 1         |
| 1.3      | Introduction . . . . .                                                     | 1         |
| 1.4      | Walkthrough Task . . . . .                                                 | 2         |
| 1.5      | Practice Tasks . . . . .                                                   | 2         |
| 1.5.1    | Practice Task 1: Create Your First Database and Table . . . . .            | 3         |
| 1.5.2    | Practice Task 2: Add and View Student Records . . . . .                    | 3         |
| 1.5.3    | Practice Task 3: Modify and Remove Student Data . . . . .                  | 3         |
| 1.6      | Unseen Tasks . . . . .                                                     | 3         |
| 1.7      | Learning Outcomes . . . . .                                                | 3         |
| 1.8      | Conclusion . . . . .                                                       | 4         |
| <b>2</b> | <b>Introduction to HTML, CSS and Forms</b>                                 | <b>5</b>  |
| 2.1      | Reference Material . . . . .                                               | 5         |
| 2.2      | Activity Timeline . . . . .                                                | 5         |
| 2.3      | Introduction . . . . .                                                     | 5         |
| 2.4      | Walkthrough Task . . . . .                                                 | 6         |
| 2.5      | Practice Tasks . . . . .                                                   | 7         |
| 2.5.1    | Practice Task 1: Create a Basic HTML Page . . . . .                        | 7         |
| 2.5.2    | Practice Task 2: Style the Webpage with CSS . . . . .                      | 7         |
| 2.5.3    | Practice Task 3: Create and Style an HTML Form . . . . .                   | 7         |
| 2.6      | Unseen Tasks . . . . .                                                     | 7         |
| 2.7      | Learning Outcomes . . . . .                                                | 7         |
| 2.8      | Conclusion . . . . .                                                       | 8         |
| <b>3</b> | <b>PHP Introduction and Form Handling</b>                                  | <b>9</b>  |
| 3.1      | Reference Material . . . . .                                               | 9         |
| 3.2      | Activity Timeline . . . . .                                                | 9         |
| 3.3      | Introduction . . . . .                                                     | 9         |
| 3.4      | Walkthrough Task . . . . .                                                 | 10        |
| 3.5      | Practice Tasks . . . . .                                                   | 10        |
| 3.5.1    | Practice Task 1: Build a Basic Form and Display User Input . . . . .       | 10        |
| 3.5.2    | Practice Task 2: Validate User Input . . . . .                             | 10        |
| 3.5.3    | Practice Task 3: Create a Contact Form with Multiple Input Types . . . . . | 10        |
| 3.6      | Unseen Tasks . . . . .                                                     | 11        |
| 3.7      | Learning Outcomes . . . . .                                                | 11        |
| 3.8      | Conclusion . . . . .                                                       | 11        |
| <b>4</b> | <b>DDL Introduction and Database Connectivity</b>                          | <b>12</b> |
| 4.1      | Reference Material . . . . .                                               | 12        |
| 4.2      | Activity Timeline . . . . .                                                | 12        |

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| 4.3      | Introduction . . . . .                                                | 12        |
| 4.4      | Walkthrough Task . . . . .                                            | 13        |
| 4.5      | Practice Tasks . . . . .                                              | 13        |
| 4.5.1    | Practice Task 1: Creating Databases and Tables . . . . .              | 13        |
| 4.5.2    | Practice Task 2: Modifying an Existing Table . . . . .                | 14        |
| 4.5.3    | Practice Task 3: Dropping Tables and Databases . . . . .              | 14        |
| 4.6      | Unseen Tasks . . . . .                                                | 14        |
| 4.7      | Learning Outcomes . . . . .                                           | 14        |
| 4.8      | Conclusion . . . . .                                                  | 14        |
| <b>5</b> | <b>Data Definition Language</b>                                       | <b>15</b> |
| 5.1      | Reference Material . . . . .                                          | 15        |
| 5.2      | Activity Timeline . . . . .                                           | 15        |
| 5.3      | Introduction . . . . .                                                | 15        |
| 5.4      | Walkthrough Task . . . . .                                            | 16        |
| 5.5      | Practice Tasks . . . . .                                              | 16        |
| 5.5.1    | Practice Task 1: Database and Table Creation . . . . .                | 16        |
| 5.5.2    | Practice Task 2: Modifying Table Structures . . . . .                 | 16        |
| 5.5.3    | Practice Task 3: Dropping Database Objects . . . . .                  | 16        |
| 5.6      | Unseen Tasks . . . . .                                                | 17        |
| 5.7      | Learning Outcomes . . . . .                                           | 17        |
| 5.8      | Conclusion . . . . .                                                  | 17        |
| <b>6</b> | <b>SQL DML: INSERT, UPDATE, DELETE and SELECT</b>                     | <b>18</b> |
| 6.1      | Reference Material . . . . .                                          | 18        |
| 6.2      | Activity Timeline . . . . .                                           | 18        |
| 6.3      | Introduction . . . . .                                                | 18        |
| 6.4      | Walkthrough Task . . . . .                                            | 19        |
| 6.5      | Practice Tasks . . . . .                                              | 19        |
| 6.5.1    | Practice Task 1: Insert Multiple Records . . . . .                    | 19        |
| 6.5.2    | Practice Task 2: Update and Modify Records . . . . .                  | 19        |
| 6.5.3    | Practice Task 3: Delete and Retrieve Specific Data . . . . .          | 19        |
| 6.6      | Unseen Tasks . . . . .                                                | 19        |
| 6.7      | Learning Outcomes . . . . .                                           | 20        |
| 6.8      | Conclusion . . . . .                                                  | 20        |
| <b>7</b> | <b>View, Update and Delete Operations Using a Web Interface</b>       | <b>21</b> |
| 7.1      | Reference Material . . . . .                                          | 21        |
| 7.2      | Activity Timeline . . . . .                                           | 21        |
| 7.3      | Introduction . . . . .                                                | 21        |
| 7.4      | Walkthrough Task . . . . .                                            | 22        |
| 7.5      | Practice Tasks . . . . .                                              | 22        |
| 7.5.1    | Practice Task 1: Create a Web-Based Student Management Page . . . . . | 22        |
| 7.5.2    | Practice Task 2: Implement Edit Functionality . . . . .               | 23        |
| 7.5.3    | Practice Task 3: Add Deletion with Confirmation . . . . .             | 23        |
| 7.6      | Unseen Tasks . . . . .                                                | 23        |
| 7.7      | Learning Outcomes . . . . .                                           | 23        |
| 7.8      | Conclusion . . . . .                                                  | 23        |

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| <b>8 SQL SELECT Functions, Advanced DML, Functions and Aggregates</b> | <b>24</b> |
| 8.1 Reference Material . . . . .                                      | 24        |
| 8.2 Activity Timeline . . . . .                                       | 24        |
| 8.3 Introduction . . . . .                                            | 24        |
| 8.4 Walkthrough Task . . . . .                                        | 25        |
| 8.5 Practice Tasks . . . . .                                          | 25        |
| 8.5.1 Practice Task 1: SELECT and Date Functions . . . . .            | 26        |
| 8.5.2 Practice Task 2: DISTINCT, Sorting and LIMIT . . . . .          | 26        |
| 8.5.3 Practice Task 3: Aggregates and Arithmetic . . . . .            | 26        |
| 8.6 Unseen Tasks . . . . .                                            | 26        |
| 8.7 Learning Outcomes . . . . .                                       | 26        |
| 8.8 Conclusion . . . . .                                              | 26        |
| <b>10 SQL GROUP BY and HAVING Clause</b>                              | <b>27</b> |
| 10.1 Reference Material . . . . .                                     | 27        |
| 10.2 Activity Timeline . . . . .                                      | 27        |
| 10.3 Introduction . . . . .                                           | 27        |
| 10.4 Walkthrough Task . . . . .                                       | 28        |
| 10.5 Practice Tasks . . . . .                                         | 28        |
| 10.5.1 Practice Task 1: GROUP BY . . . . .                            | 28        |
| 10.5.2 Practice Task 2: HAVING Clause . . . . .                       | 28        |
| 10.5.3 Practice Task 3: Advanced Grouping . . . . .                   | 28        |
| 10.6 Unseen Tasks . . . . .                                           | 29        |
| 10.7 Learning Outcomes . . . . .                                      | 29        |
| 10.8 Conclusion . . . . .                                             | 29        |
| <b>11 SQL JOINS: INNER JOIN and SELF JOIN</b>                         | <b>30</b> |
| 11.1 Reference Material . . . . .                                     | 30        |
| 11.2 Activity Timeline . . . . .                                      | 30        |
| 11.3 Introduction . . . . .                                           | 30        |
| 11.4 Walkthrough Task . . . . .                                       | 31        |
| 11.5 Practice Tasks . . . . .                                         | 31        |
| 11.5.1 Practice Task 1: Basic INNER JOIN . . . . .                    | 31        |
| 11.5.2 Practice Task 2: Moderate INNER JOIN . . . . .                 | 31        |
| 11.5.3 Practice Task 3: SELF JOIN . . . . .                           | 32        |
| 11.6 Unseen Tasks . . . . .                                           | 32        |
| 11.7 Learning Outcomes . . . . .                                      | 32        |
| 11.8 Conclusion . . . . .                                             | 32        |
| <b>13 Introduction to MongoDB: Installation and CRUD Operations</b>   | <b>33</b> |
| 13.1 Reference Material . . . . .                                     | 33        |
| 13.2 Software Required . . . . .                                      | 33        |
| 13.3 Introduction . . . . .                                           | 33        |
| 13.4 Walkthrough Task . . . . .                                       | 34        |
| 13.5 Practice Tasks . . . . .                                         | 34        |
| 13.5.1 Practice Task 1: Build a Practice Database . . . . .           | 34        |
| 13.5.2 Practice Task 2: Retrieve and Filter Documents . . . . .       | 34        |
| 13.5.3 Practice Task 3: Update and Delete Documents . . . . .         | 35        |

---

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| 13.6 Unseen Tasks . . . . .                                           | 35        |
| 13.7 Learning Outcomes . . . . .                                      | 35        |
| 13.8 Conclusion . . . . .                                             | 35        |
| <b>14 MongoDB Advanced: Update, Delete and Aggregation Operations</b> | <b>36</b> |
| 14.1 Reference Material . . . . .                                     | 36        |
| 14.2 Software Required . . . . .                                      | 36        |
| 14.3 Introduction . . . . .                                           | 36        |
| 14.4 Walkthrough Task . . . . .                                       | 36        |
| 14.5 Practice Tasks . . . . .                                         | 37        |
| 14.5.1 Practice Task 1: Update Operations . . . . .                   | 37        |
| 14.5.2 Practice Task 2: Delete Operations . . . . .                   | 37        |
| 14.5.3 Practice Task 3: Aggregation Operations . . . . .              | 37        |
| 14.6 Unseen Tasks . . . . .                                           | 38        |
| 14.7 Learning Outcomes . . . . .                                      | 38        |
| 14.8 Conclusion . . . . .                                             | 38        |

---

# Introduction to SQL Tools and Techniques

---

## Lab Objectives

1. Install and configure MySQL Server and MySQL Workbench for database management.
2. Become familiar with the SQL development environment and its core features.
3. Execute basic SQL operations for creating, inserting, retrieving, updating, and deleting data.

## 1.1 Reference Material

- MySQL Documentation: <https://dev.mysql.com/doc/>
- MySQL Installer for Windows: <https://dev.mysql.com/downloads/installer/>
- TutorialsPoint SQL Guide: <https://www.tutorialspoint.com/sql/index.htm>
- W3Schools SQL Tutorial: <https://www.w3schools.com/sql/>

## 1.2 Activity Timeline

| Task No. | Activity                            | Time Allocation |
|----------|-------------------------------------|-----------------|
| 1        | Introduction and guided walkthrough | 20 minutes      |
| 2        | Practice Task 01                    | 40 minutes      |
| 3        | Practice Task 02                    | 40 minutes      |
| 4        | Practice Task 03                    | 40 minutes      |
| 5        | Conclusion and wrap-up              | 20 minutes      |

## 1.3 Introduction

A database is an organized collection of related data that can be stored, queried, updated, and managed efficiently. Relational database management systems such as MySQL organize data into tables consisting of rows and columns. The Structured Query Language (SQL) provides a standard way to define structures, manipulate records, and retrieve useful information from these tables.

This lab introduces MySQL Server and MySQL Workbench as the main tools used throughout the course. Students first configure a working database environment and then perform the essential operations that form the basis of later database development. Particular attention is given to

understanding the difference between the database server, which stores and processes data, and the graphical client, which allows users to write and execute SQL statements more conveniently.

### Key Concept

MySQL Server is the database engine. MySQL Workbench is a client application used to connect to the server, create schemas, run queries, inspect tables, and administer database objects.

## 1.4 Walkthrough Task

The walkthrough guides students through the complete setup and first use of MySQL. Begin by downloading MySQL Server and MySQL Workbench from the official website. During installation, ensure the server component and Workbench are selected, retain an appropriate port configuration, and create a secure root password. After installation, create a new connection in Workbench and test it to verify that the server is reachable.

Once connected, create a database for student records and define a simple table. Then insert sample rows, retrieve the complete table, filter selected records, update an existing value, and delete a chosen record. The following example illustrates the sequence of operations.

Listing 1.1: Basic SQL workflow

```
CREATE DATABASE StudentRecords;
USE StudentRecords;

CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Age INT,
    Department VARCHAR(80),
    EnrollmentDate DATE
);

INSERT INTO Students
(StudentID, Name, Age, Department, EnrollmentDate)
VALUES
(1, 'Ali Khan', 20, 'Software Engineering', '2025-02-10'),
(2, 'Sara Ahmed', 21, 'Computer Science', '2025-02-12');

SELECT * FROM Students;

UPDATE Students
SET Department = 'Artificial Intelligence'
WHERE StudentID = 2;

DELETE FROM Students
WHERE StudentID = 1;
```

## 1.5 Practice Tasks

### 1.5.1 Practice Task 1: Create Your First Database and Table

- Create a new database dedicated to student records.
- Create a table containing ID, name, age, department, and enrollment date.
- Select suitable data types and define the ID as a primary key.
- Verify the table structure using the MySQL Workbench schema panel or a suitable SQL command.

### 1.5.2 Practice Task 2: Add and View Student Records

- Insert at least five realistic student records.
- Retrieve all records from the table.
- Write filtered queries based on department, age, or enrollment date.
- Display only selected columns to understand projection in SQL.

### 1.5.3 Practice Task 3: Modify and Remove Student Data

- Update the age or department of one student.
- Delete a student record using a unique ID or another precise condition.
- Display the updated table to confirm the changes.
- Explain why a `WHERE` clause is important when using `UPDATE` and `DELETE`.

## 1.6 Unseen Tasks

1. Create a new database named `LibraryDB` and a table named `Books` with suitable fields for book ID, title, author, category, and publication year. Insert at least five records and display the complete table.
2. Write SQL statements to update the category of one selected book and delete another book using its unique ID. Verify the table after both operations.
3. Retrieve only books published after a year of your choice and display the title, author, and publication year in ascending order of title.

## 1.7 Learning Outcomes

By the end of this lab, students will be able to install and configure MySQL tools, create databases and tables, execute introductory SQL statements, and safely perform basic data manipulation. They will also be able to distinguish between database structure and database content, an important foundation for later DDL and DML work.

## 1.8 Conclusion

This lab established the practical foundation for working with relational databases. Students configured MySQL Server and Workbench, created a schema and table, and practiced the basic SQL operations needed to manage records. These skills prepare students for more advanced topics including data definition, constraints, query design, application connectivity, and database-backed web systems.

---

# Introduction to HTML, CSS and Forms

---

## Lab Objectives

- Understand the basic structure of an HTML document and commonly used tags.
- Apply CSS rules to improve the layout and presentation of web pages.
- Create HTML forms to collect user input.
- Integrate content structure, presentation, and input controls in a simple web interface.

## 2.1 Reference Material

- W3Schools HTML Tutorial: <https://www.w3schools.com/html/>
- W3Schools CSS Tutorial: <https://www.w3schools.com/css/>
- MDN Web Docs: <https://developer.mozilla.org/>
- Bootstrap: <https://getbootstrap.com/>

## 2.2 Activity Timeline

| Task No. | Activity                            | Time Allocation |
|----------|-------------------------------------|-----------------|
| 1        | Introduction and guided walkthrough | 20 minutes      |
| 2        | Practice Task 01                    | 40 minutes      |
| 3        | Practice Task 02                    | 40 minutes      |
| 4        | Practice Task 03                    | 40 minutes      |
| 5        | Conclusion                          | 20 minutes      |

## 2.3 Introduction

Modern database applications are often accessed through a web interface. HTML defines the structure of a page, CSS controls its appearance, and forms provide the input controls needed to collect information from users. Although HTML and CSS do not directly manipulate a database, they provide the front-end layer through which users interact with server-side code and database services.

HTML uses elements such as headings, paragraphs, lists, links, images, tables, and forms. CSS separates presentation from content by defining fonts, spacing, borders, colors, and page layout. Forms add interactive controls such as text fields, radio buttons, checkboxes, drop-down menus, and

submit buttons. Understanding how these three areas work together is essential before introducing PHP-based form processing and database connectivity.

## 2.4 Walkthrough Task

Create a small student registration page. Start with the standard HTML document structure, add a page heading and explanatory paragraph, and then build a form containing name, email, program, and semester fields. Apply CSS using an external stylesheet so that visual formatting remains separate from the page content.

Listing 2.1: Simple HTML form

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Student Registration</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="container">
    <h1>Student Registration</h1>
    <form method="post" action="process.php">
      <label>Name</label>
      <input type="text" name="name" required>

      <label>Email</label>
      <input type="email" name="email" required>

      <label>Program</label>
      <select name="program">
        <option>BS Software Engineering</option>
        <option>BS Computer Science</option>
      </select>

      <button type="submit">Submit</button>
    </form>
  </main>
</body>
</html>
```

Listing 2.2: Basic CSS styling

```
body {
  font-family: Arial, sans-serif;
  background: #f4f6f8;
}
.container {
  width: 70%;
  margin: 40px auto;
  padding: 24px;
  background: white;
  border-radius: 8px;
```

```
}  
label, input, select, button {  
    display: block;  
    width: 100%;  
    margin-bottom: 12px;  
}
```

## 2.5 Practice Tasks

### 2.5.1 Practice Task 1: Create a Basic HTML Page

Build a valid HTML page containing headings, paragraphs, ordered or unordered lists, at least one hyperlink, and one image. Organize the page using meaningful structural elements rather than placing all content inside generic containers.

### 2.5.2 Practice Task 2: Style the Webpage with CSS

Apply inline and internal CSS briefly for comparison, then move the final formatting to an external stylesheet. Use properties related to typography, spacing, borders, backgrounds, and alignment. The final page should be readable, consistent, and visually balanced.

### 2.5.3 Practice Task 3: Create and Style an HTML Form

Design a form that contains text input, email input, radio buttons, checkboxes, a text area, and a submit button. Ensure labels are clear, input controls are logically grouped, and required fields are indicated. Apply CSS so that the form can later serve as a practical front end for database operations.

## 2.6 Unseen Tasks

1. Design a one-page student registration interface using semantic HTML. Include fields for name, email, program, semester, gender, and an agreement checkbox.
2. Create an external CSS file that formats the registration page using a centered container, consistent spacing, styled form controls, and a responsive layout suitable for smaller screens.
3. Build a second page containing a course feedback form with radio buttons, checkboxes, a rating selection, and a comments area. Apply the same external stylesheet to maintain a consistent visual design.

## 2.7 Learning Outcomes

Students will be able to build structured HTML pages, apply CSS styling consistently, create forms for collecting data, and explain how the front-end interface connects conceptually to future PHP and MySQL work.

## 2.8 Conclusion

This lab introduced the core web technologies needed to create a usable interface for database applications. Students practiced separating structure from presentation and learned how forms capture user input. The skills developed here directly support the next lab, where PHP will process form data and produce dynamic output.

---

# PHP Introduction and Form Handling

---

## Lab Objectives

1. Understand the interaction between HTML forms and PHP scripts.
2. Learn PHP syntax, variables, conditional statements, loops, and arrays.
3. Process and validate user input submitted through forms.
4. Build simple interactive PHP applications as preparation for database integration.

## 3.1 Reference Material

- Welling, L., & Thomson, L. (2009). *PHP and MySQL Web Development*. Addison-Wesley.
- PHP Official Documentation: <https://www.php.net/docs.php>
- W3Schools PHP Tutorial: <https://www.w3schools.com/php/>
- TutorialsPoint PHP Guide: <https://www.tutorialspoint.com/php/>

## 3.2 Activity Timeline

| Task No. | Activity                            | Time Allocation |
|----------|-------------------------------------|-----------------|
| 1        | Introduction and guided walkthrough | 20 minutes      |
| 2        | Practice Task 01                    | 40 minutes      |
| 3        | Practice Task 02                    | 40 minutes      |
| 4        | Practice Task 03                    | 40 minutes      |
| 5        | Conclusion and discussion           | 20 minutes      |

## 3.3 Introduction

PHP is a server-side scripting language widely used to generate dynamic web pages and process user input. Unlike HTML, which is delivered directly to the browser, PHP code executes on the server and produces output that is then sent to the client. This makes PHP suitable for tasks such as validating form submissions, managing sessions, connecting to databases, and controlling application logic.

A central focus of this lab is the use of the `GET` and `POST` methods. `GET` appends submitted values to the URL and is suitable for non-sensitive retrieval-oriented operations. `POST` sends values in the request body and is generally preferred for form submissions that create or modify data. Students

should also recognize that server-side validation remains necessary even when HTML validation attributes are present.

## 3.4 Walkthrough Task

Create a simple login form and process it with PHP. The form collects a username and password. The PHP script checks whether fields are empty, compares the supplied values with predefined credentials, and displays either a success or error message. The purpose is to understand request handling and control flow rather than implement production-grade authentication.

Listing 3.1: Basic PHP form processing

```
<?php
$message = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $username = trim($_POST["username"] ?? "");
    $password = $_POST["password"] ?? "";

    if ($username === "" || $password === "") {
        $message = "Please complete all fields.";
    } elseif ($username === "student" && $password === "db123") {
        $message = "Login successful.";
    } else {
        $message = "Invalid username or password.";
    }
}
?>
```

### Security Note

Real applications should never store plain-text passwords or rely on hard-coded credentials. Passwords should be hashed using appropriate password functions, and all user-controlled input should be validated before use.

## 3.5 Practice Tasks

### 3.5.1 Practice Task 1: Build a Basic Form and Display User Input

Create a form that collects a name and email address. Submit the form to a PHP script using POST, retrieve the values, sanitize or validate them, and display a personalized confirmation message.

### 3.5.2 Practice Task 2: Validate User Input

Extend the form by checking that required fields are not empty and that the email address has a valid format. Display field-specific messages when validation fails and a confirmation message when all checks pass.

### 3.5.3 Practice Task 3: Create a Contact Form with Multiple Input Types

Create a contact form containing name, email, gender selection, optional interests, and a message field. Process the data with PHP and display a well-formatted summary. Ensure optional fields are

handled gracefully when no value is submitted.

### 3.6 Unseen Tasks

1. Create a PHP form that accepts a student's name and three subject marks, validates all fields, calculates the total and percentage, and displays a suitable grade.
2. Build a PHP form that accepts a positive integer and displays its multiplication table from 1 to 10 using a loop. Reject empty, non-numeric, or negative input.
3. Create a small contact form containing name, email, subject, and message fields. Process the form with PHP and display a formatted summary only when all required input is valid.

### 3.7 Learning Outcomes

After this lab, students will be able to write introductory PHP code, process HTML form submissions, apply conditional logic and validation, and explain the difference between client-side and server-side processing.

### 3.8 Conclusion

This lab connected static front-end forms with server-side processing. Students practiced PHP fundamentals and used them to validate and respond to user input. These concepts are essential for the next stage of the course, where PHP will connect to MySQL and perform operations against persistent database records.

# DDL Introduction and Database Connectivity

## Lab Objectives

1. Create and manage databases and tables using Data Definition Language (DDL).
2. Modify and remove database structures safely.
3. Connect PHP applications with MySQL for dynamic data interaction.
4. Execute SQL statements that reflect practical database management requirements.

## 4.1 Reference Material

- Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems*. Pearson.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts*. McGraw-Hill.
- MySQL Documentation: <https://dev.mysql.com/doc/>
- W3Schools MySQL Tutorial: <https://www.w3schools.com/mysql/>
- PHP MySQLi Manual: <https://www.php.net/manual/en/book.mysqli.php>

## 4.2 Activity Timeline

| Task No. | Activity                            | Time Allocation |
|----------|-------------------------------------|-----------------|
| 1        | Introduction and guided walkthrough | 20 minutes      |
| 2        | Practice Task 01                    | 40 minutes      |
| 3        | Practice Task 02                    | 40 minutes      |
| 4        | Practice Task 03                    | 40 minutes      |
| 5        | Conclusion and discussion           | 20 minutes      |

## 4.3 Introduction

Data Definition Language (DDL) contains SQL statements used to define, alter, and remove database structures. Common DDL commands include **CREATE**, **ALTER**, and **DROP**. These commands affect schemas and table definitions rather than the individual rows stored inside the tables. Because structural changes can have significant consequences, DDL operations should be performed deliberately and, in real systems, with appropriate backups and change-control procedures.

This lab also introduces application-to-database connectivity. A web application needs a secure and reliable way to connect to MySQL before it can retrieve or modify data. PHP can establish this connection using MySQLi or PDO. The connection typically requires a server address, username, password, and database name. Error handling should be included so that failures can be detected without exposing sensitive system details to end users.

## 4.4 Walkthrough Task

Create a database named `StudentPortal` and a `Students` table. Add columns for student identity and contact information, then use `ALTER TABLE` to extend or modify the schema. Finally, create a simple PHP connection script and verify that the application can connect successfully.

Listing 4.1: DDL example

```
CREATE DATABASE StudentPortal;
USE StudentPortal;

CREATE TABLE Students (
    StudentID INT AUTO_INCREMENT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Email VARCHAR(120) UNIQUE NOT NULL
);

ALTER TABLE Students
ADD COLUMN Department VARCHAR(80);
```

Listing 4.2: PHP to MySQL connection with MySQLi

```
<?php
$host = "localhost";
$user = "root";
$pass = "your_password";
$db = "StudentPortal";

$conn = new mysqli($host, $user, $pass, $db);

if ($conn->connect_error) {
    die("Database connection failed.");
}

echo "Connected successfully.";
?>
```

## 4.5 Practice Tasks

### 4.5.1 Practice Task 1: Creating Databases and Tables

Create `CollegeDB` and a `Courses` table containing `CourseID`, `CourseName`, `Instructor`, and `CreditHours`. Apply appropriate constraints such as `PRIMARY KEY`, `AUTO_INCREMENT`, and `NOT NULL`. Verify the resulting table structure.

### 4.5.2 Practice Task 2: Modifying an Existing Table

Use `ALTER TABLE` to add a new column, rename a column, change a data type, and remove a column. Before each operation, identify whether existing data could be affected or lost.

### 4.5.3 Practice Task 3: Dropping Tables and Databases

Drop the `Courses` table and then remove `CollegeDB`. Explain why destructive commands must be verified carefully and why database backups are important in real production environments.

## 4.6 Unseen Tasks

1. Create a database named `InventoryDB` with a `Products` table containing product ID, product name, category, unit price, and stock quantity. Apply suitable data types and constraints.
2. Alter the `Products` table by adding a supplier field, changing the size of the product-name field, and removing a field that is no longer required. Verify the structure after each change.
3. Create a simple PHP connection file for `InventoryDB` and display a clear browser message for successful and unsuccessful database connections.

## 4.7 Learning Outcomes

Students will be able to define and modify database structures, use common integrity constraints, connect a PHP application to MySQL, and describe practical precautions associated with schema changes.

## 4.8 Conclusion

This lab strengthened students' understanding of database structure by combining DDL commands with introductory application connectivity. Students practiced creating, altering, and removing database objects, then connected PHP with MySQL. These skills bridge database design and application development and prepare students for more structured DDL and DML work.

---

# Data Definition Language

---

## Lab Objectives

1. Apply **CREATE**, **ALTER**, and **DROP** to define and manage database structures.
2. Design relational tables with suitable data types, primary keys, and integrity constraints.
3. Use MySQL Workbench effectively to execute DDL and verify structural changes.

## 5.1 Reference Material

- Silberschatz, A., Korth, H. F., & Sudarshan, S. *Database System Concepts*. McGraw-Hill Education.
- Coronel, C., & Morris, S. *Database Systems: Design, Implementation, & Management*. Cengage Learning.
- MySQL Documentation: <https://dev.mysql.com/doc/>

## 5.2 Activity Timeline

| Task No. | Activity                     | Time Allocation |
|----------|------------------------------|-----------------|
| 1        | Introduction and walkthrough | 20 minutes      |
| 2        | Practice Task 01             | 50 minutes      |
| 3        | Practice Task 02             | 50 minutes      |
| 4        | Practice Task 03             | 40 minutes      |
| 5        | Conclusion and wrap-up       | 20 minutes      |

## 5.3 Introduction

DDL defines the logical structure of a relational database. A well-designed schema determines what data can be stored, how it is represented, and which rules protect its consistency. Choosing appropriate data types reduces storage waste and prevents invalid values. Primary keys uniquely identify rows, while constraints such as **NOT NULL**, **UNIQUE**, **DEFAULT**, and **FOREIGN KEY** enforce important business and relational rules.

The emphasis in this lab is not only on writing syntactically correct commands but also on thinking about design consequences. For example, changing a column from a wide string to a smaller type may cause truncation, while dropping a table can permanently remove both its definition and stored

data. Students should therefore connect each DDL statement with its effect on the schema and existing records.

## 5.4 Walkthrough Task

Create a school database with multiple related tables. Define keys and constraints, inspect the generated schema, and then modify selected columns. The sample below demonstrates a simple parent-child relationship.

Listing 5.1: Creating related tables with constraints

```
CREATE DATABASE SchoolDB;
USE SchoolDB;

CREATE TABLE Teachers (
    TeacherID INT AUTO_INCREMENT PRIMARY KEY,
    FullName VARCHAR(100) NOT NULL,
    Email VARCHAR(120) UNIQUE
);

CREATE TABLE Students (
    StudentID INT AUTO_INCREMENT PRIMARY KEY,
    FullName VARCHAR(100) NOT NULL,
    Age INT,
    AdvisorID INT,
    CONSTRAINT fk_advisor
        FOREIGN KEY (AdvisorID)
        REFERENCES Teachers(TeacherID)
);
```

## 5.5 Practice Tasks

### 5.5.1 Practice Task 1: Database and Table Creation

Create `SchoolDB` and define `Students` and `Teachers` tables. Choose meaningful attributes and appropriate data types. Include primary keys and at least two additional integrity constraints. Explain how each constraint improves data quality.

### 5.5.2 Practice Task 2: Modifying Table Structures

Add a new attribute, rename an existing column, change a column data type, and remove an obsolete attribute. After every command, verify the updated table definition in MySQL Workbench.

### 5.5.3 Practice Task 3: Dropping Database Objects

Remove one table and then delete the full database after confirming that the exercise is complete. Document the commands used and state one precaution that should be taken before performing each destructive operation in a real system.

## 5.6 Unseen Tasks

1. Create a database named **ClinicDB** with **Patients** and **Doctors** tables. Define suitable primary keys, mandatory fields, and meaningful data types.
2. Modify the **Patients** table by adding a contact number field, changing the size of the patient-name field, and renaming one column to reflect an updated requirement.
3. After verifying the schema, remove one selected table and finally drop the complete practice database. Record the SQL commands used for each structural operation.

## 5.7 Learning Outcomes

Students will be able to create relational schemas, select suitable data types and constraints, alter table definitions in response to changing requirements, and safely remove database objects when required.

## 5.8 Conclusion

This lab provided focused practice in DDL operations. Students designed relational structures, applied integrity constraints, modified schemas, and removed database objects. The exercise reinforces the idea that schema design directly affects data integrity, maintainability, and the reliability of future application logic.

# SQL DML: INSERT, UPDATE, DELETE and SELECT

## Lab Objectives

1. Use `INSERT` to add records accurately.
2. Apply `UPDATE` and `DELETE` to modify or remove existing data safely.
3. Execute `SELECT` queries to retrieve and display records based on conditions.

## 6.1 Reference Material

- Silberschatz, A., Korth, H. F., & Sudarshan, S. *Database System Concepts*. McGraw-Hill Education.
- Coronel, C., & Morris, S. *Database Systems: Design, Implementation, & Management*. Cengage Learning.
- MySQL Documentation: <https://dev.mysql.com/doc/>

## 6.2 Activity Timeline

| Task No. | Activity                            | Time Allocation |
|----------|-------------------------------------|-----------------|
| 1        | Introduction and guided walkthrough | 20 minutes      |
| 2        | Practice Task 01                    | 40 minutes      |
| 3        | Practice Task 02                    | 40 minutes      |
| 4        | Practice Task 03                    | 40 minutes      |
| 5        | Conclusion and wrap-up              | 20 minutes      |

## 6.3 Introduction

Data Manipulation Language (DML) is used to work with the records stored inside database tables. Whereas DDL defines the structure of a database, DML changes or retrieves the data contained within that structure. The main commands introduced in this lab are `INSERT`, `UPDATE`, `DELETE`, and `SELECT`.

Safe DML requires careful use of conditions. An `UPDATE` statement without a `WHERE` clause may modify every row, while a `DELETE` statement without a condition may remove all records. Students

should therefore verify selection criteria before executing modifications and, when possible, first run a corresponding **SELECT** query to confirm the rows that will be affected.

## 6.4 Walkthrough Task

Insert several rows into an existing table, retrieve them, modify selected values, remove a target row, and execute filtered queries. Include both row filtering and column selection so that students understand how a query can limit results vertically and horizontally.

Listing 6.1: Core DML examples

```
INSERT INTO Students (FullName, Age, Email)
VALUES
('Ayesha Malik', 20, 'ayesha@example.com'),
('Hamza Tariq', 21, 'hamza@example.com');

SELECT StudentID, FullName, Email
FROM Students
WHERE Age >= 21;

UPDATE Students
SET Email = 'hamza.tariq@example.com'
WHERE StudentID = 2;

DELETE FROM Students
WHERE StudentID = 1;
```

## 6.5 Practice Tasks

### 6.5.1 Practice Task 1: Insert Multiple Records

Insert at least five students with meaningful values for **StudentID**, **Name**, **EnrollmentDate**, and **Email**. Retrieve the table after insertion and verify that no required fields were omitted.

### 6.5.2 Practice Task 2: Update and Modify Records

Update the email address of one student based on **StudentID**. Then update the enrollment date for a logically selected group of students. Before running each **UPDATE**, write a **SELECT** query using the same condition to verify which rows will be affected.

### 6.5.3 Practice Task 3: Delete and Retrieve Specific Data

Delete students who no longer attend the institution using a precise condition. Retrieve students who enrolled after a specified date and display only the **Name** and **Email** columns. Sort the result to improve readability.

## 6.6 Unseen Tasks

1. Insert at least six records into a **Products** table containing product ID, name, category, price, and stock. Retrieve products with a price above a chosen threshold.

2. Update the stock quantity of selected products and increase the price of one category by 5%. Use a **SELECT** statement before each update to verify the rows that will be affected.
3. Delete products with zero stock and then display only product name, category, and price for the remaining records, ordered by price from highest to lowest.

## 6.7 Learning Outcomes

Students will be able to insert new records, update existing data, delete selected rows, and retrieve targeted information using conditional **SELECT** statements. They will also understand the practical importance of safe filtering when changing data.

## 6.8 Conclusion

This lab developed the core data-handling skills required for day-to-day database work. Students used DML commands to add, change, remove, and retrieve records in MySQL. These operations form the foundation for CRUD-based applications and will be used directly in the final web-interface lab.

---

# View, Update and Delete Operations Using a Web Interface

---

## Lab Objectives

1. Display database records dynamically on a web page using server-side scripting.
2. Create web forms that update existing records with validation and secure processing.
3. Implement controlled record deletion from a web interface with appropriate user confirmation.

## 7.1 Reference Material

- Welling, L., & Thomson, L. *PHP and MySQL Web Development*. Addison-Wesley.
- Ullman, L. *PHP and MySQL for Dynamic Web Sites*. Peachpit Press.
- PHP Documentation: <https://www.php.net/docs.php>
- MySQL Documentation: <https://dev.mysql.com/doc/>

## 7.2 Activity Timeline

| Task No. | Activity                            | Time Allocation |
|----------|-------------------------------------|-----------------|
| 1        | Introduction and guided walkthrough | 20 minutes      |
| 2        | Practice Task 01                    | 40 minutes      |
| 3        | Practice Task 02                    | 40 minutes      |
| 4        | Practice Task 03                    | 40 minutes      |
| 5        | Conclusion and wrap-up              | 20 minutes      |

## 7.3 Introduction

Most end users do not interact with a database by writing SQL commands directly. Instead, applications provide forms, tables, buttons, and navigation controls that translate user actions into server-side database operations. This lab integrates the technologies introduced earlier by connecting an HTML/PHP web interface to MySQL and implementing view, update, and delete functionality.

The exercise reflects three parts of the CRUD model: read, update, and delete. Data is read from the database and displayed in an HTML table; update forms allow selected fields to be edited; and delete controls remove records after confirmation. The lab also introduces the idea that application

code should validate input and use safe query techniques such as prepared statements to reduce the risk of SQL injection.

### Recommended Practice

Use prepared statements for values supplied by users. Do not build SQL by directly concatenating untrusted form or URL input into query strings.

## 7.4 Walkthrough Task

Prepare a local server environment such as XAMPP, WAMP, or MAMP and ensure a **StudentDB** database contains a **students** table with sample records. Create a main page that retrieves all records and displays them in an HTML table with *Edit* and *Delete* actions. The edit page should load one record by ID, pre-fill a form, validate the submitted data, and save the changes. The delete action should require confirmation and remove only the intended record.

Listing 7.1: Displaying records using PHP and MySQLi

```
<?php
$result = $conn->query(
    "SELECT StudentID, FullName, Email FROM Students"
);

while ($row = $result->fetch_assoc()) {
    echo "<tr>";
    echo "<td>" . htmlspecialchars($row['StudentID']) . "</td>";
    echo "<td>" . htmlspecialchars($row['FullName']) . "</td>";
    echo "<td>" . htmlspecialchars($row['Email']) . "</td>";
    echo "</tr>";
}
?>
```

Listing 7.2: Prepared statement for an update

```
<?php
$stmt = $conn->prepare(
    "UPDATE Students SET FullName = ?, Email = ? WHERE StudentID = ?"
);
$stmt->bind_param("ssi", $name, $email, $id);
$stmt->execute();
?>
```

## 7.5 Practice Tasks

### 7.5.1 Practice Task 1: Create a Web-Based Student Management Page

Connect a PHP page to **StudentDB**, retrieve all records from the **students** table, and display them in a clear HTML table. Add an *Edit* and *Delete* action for each row. Use consistent styling and escape displayed values appropriately.

### 7.5.2 Practice Task 2: Implement Edit Functionality

Load a student record using an ID supplied through the URL. Pre-fill a form with current values, allow changes, validate the submitted data, and update the database using a prepared statement. Redirect back to the main page with a clear success or error message.

### 7.5.3 Practice Task 3: Add Deletion with Confirmation

Implement deletion based on a unique student ID. Add a confirmation step to reduce accidental deletion and use a prepared statement to ensure the intended record is targeted. Redirect to the listing page and display a confirmation message after successful deletion.

## 7.6 Unseen Tasks

1. Extend the student management interface with a search form that retrieves students by partial name or email and displays matching records in the existing table layout.
2. Add a web form for inserting a new student record. Validate required fields, use a prepared statement, and redirect to the main listing after a successful insertion.
3. Add a status field to the student table and create an interface control that allows a user to mark a student as Active or Inactive without deleting the record.

## 7.7 Learning Outcomes

By the end of the lab, students will be able to integrate PHP with MySQL, display live database content in a browser, update records through forms, and delete records with confirmation and safer database interaction techniques.

## 7.8 Conclusion

This lab brought together the main technologies introduced throughout the manual. Students connected a web interface to a MySQL database and implemented practical view, edit, and delete operations. The exercise demonstrates how SQL, PHP, HTML, and CSS combine to form a basic full-stack, data-driven application while reinforcing the importance of validation, safe query execution, and clear user feedback.

---

# SQL SELECT Functions, Advanced DML, Functions and Aggregates

---

## Lab Objectives

1. Apply date and time functions including `CURDATE()`, `NOW()`, `DATEDIFF()`, `YEAR()`, and `MONTH()`.
2. Use `ORDER BY`, `DISTINCT`, and `LIMIT` to refine query results.
3. Apply text and numeric functions such as `UPPER()`, `LOWER()`, `LENGTH()`, `ROUND()`, `CEIL()`, `FLOOR()`, and `CONCAT()`.
4. Use aggregate functions such as `COUNT()`, `SUM()`, `AVG()`, `MIN()`, and `MAX()`.
5. Perform arithmetic calculations inside SQL queries to produce computed values.

## 8.1 Reference Material

- MySQL Documentation: <https://dev.mysql.com/doc/>
- W3Schools SQL Tutorial: <https://www.w3schools.com/sql/>
- TutorialsPoint SQL Guide: <https://www.tutorialspoint.com/sql/>

## 8.2 Activity Timeline

| Task No. | Activity                            | Time Allocation |
|----------|-------------------------------------|-----------------|
| 1        | Introduction and guided walkthrough | 20 minutes      |
| 2        | Practice Task 01                    | 40 minutes      |
| 3        | Practice Task 02                    | 40 minutes      |
| 4        | Practice Task 03                    | 40 minutes      |
| 5        | Conclusion and wrap-up              | 20 minutes      |

## 8.3 Introduction

This lab develops analytical SQL skills by combining built-in functions, sorting, duplicate elimination, result limiting, aggregate calculations, and arithmetic expressions. These features make it possible to transform raw table values into meaningful information directly within a query. Students move beyond simple retrieval and learn how SQL can format text, work with dates, summarize large datasets, and create calculated output fields.

Date functions are especially useful for enrollment, attendance, billing, and transaction data. Text functions help standardize output, while numeric functions are useful for rounding and calculations. Aggregate functions summarize multiple rows into a single result, allowing users to answer questions such as how many records exist, what the average value is, or which value is highest or lowest.

## 8.4 Walkthrough Task

Begin with an existing `Students` table containing student names, ages, departments, enrollment dates, fees, and marks. Execute a sequence of queries that demonstrates string transformation, date extraction, sorting, duplicate elimination, limiting results, aggregate calculations, and computed columns.

Listing 8.1: String and date functions

```
SELECT Name,
       UPPER(Name) AS UpperName,
       LOWER(Name) AS LowerName,
       LENGTH(Name) AS NameLength,
       CONCAT(Name, ' - ', Department) AS FullInfo
FROM Students;

SELECT Name,
       EnrollmentDate,
       YEAR(EnrollmentDate) AS YearEnrolled,
       MONTH(EnrollmentDate) AS MonthEnrolled,
       DATEDIFF(CURDATE(), EnrollmentDate) AS DaysSinceEnrollment
FROM Students;
```

Listing 8.2: Sorting, limiting, aggregates, and arithmetic

```
SELECT DISTINCT Department
FROM Students;

SELECT *
FROM Students
ORDER BY EnrollmentDate DESC
LIMIT 5;

SELECT COUNT(*) AS TotalStudents,
       AVG(Age) AS AverageAge,
       MIN(Age) AS Youngest,
       MAX(Age) AS Oldest
FROM Students;

SELECT Name, Age, Age * 12 AS AgeInMonths
FROM Students;
```

## 8.5 Practice Tasks

### 8.5.1 Practice Task 1: SELECT and Date Functions

Display each student's name in uppercase and lowercase, show the length of the name, extract the enrollment year and month, calculate the number of days since enrollment, and combine the student's name and department into a single output field.

### 8.5.2 Practice Task 2: DISTINCT, Sorting and LIMIT

List all unique departments, display the three most recent enrollments, sort students alphabetically by name, and display records ordered from newest to oldest enrollment date.

### 8.5.3 Practice Task 3: Aggregates and Arithmetic

Calculate the total number of students, minimum age, maximum age, and average age. Display the number of students in each department, convert age into months, calculate a 10% discount on a **Fee** field, and compute a percentage from **MarksObtained** and **TotalMarks**.

## 8.6 Unseen Tasks

1. Write a query that displays each student's name in title-related output by combining the department and enrollment year, then sort the result by enrollment year in descending order and return only the first five rows.
2. Using a table containing **Fee**, calculate the original fee, a 15% scholarship amount, and the final payable fee for every student. Round the final amount to two decimal places.
3. Produce a department-wise summary showing the total number of students, average age, youngest age, and oldest age, and order the departments by the highest student count.

## 8.7 Learning Outcomes

By the end of this lab, students will be able to transform textual, numeric, and date values using SQL functions; refine output with **DISTINCT**, **ORDER BY**, and **LIMIT**; perform calculations in queries; and summarize data through aggregate functions.

## 8.8 Conclusion

This lab strengthened the transition from basic SQL retrieval to analytical query writing. Students used built-in functions to transform data, applied sorting and limiting to control output, and generated numerical summaries using aggregates and arithmetic expressions. These skills are essential for reporting, dashboards, and later work with grouped and joined datasets.

# SQL GROUP BY and HAVING Clause

## Lab Objectives

1. Understand how **GROUP BY** organizes rows into logical groups for aggregation.
2. Apply aggregate functions with **GROUP BY** to generate grouped summaries.
3. Use **HAVING** to filter grouped results using aggregate conditions.
4. Develop analytical SQL queries for reporting and dataset segmentation.

## 10.1 Reference Material

- W3Schools SQL GROUP BY Tutorial: [https://www.w3schools.com/sql/sql\\_groupby.asp](https://www.w3schools.com/sql/sql_groupby.asp)
- MySQL Documentation: <https://dev.mysql.com/doc/>
- TutorialsPoint SQL HAVING Clause: <https://www.tutorialspoint.com/sql/sql-having-clause.htm>

## 10.2 Activity Timeline

| Task No. | Activity                            | Time Allocation |
|----------|-------------------------------------|-----------------|
| 1        | Introduction and guided walkthrough | 20 minutes      |
| 2        | Practice Task 01                    | 40 minutes      |
| 3        | Practice Task 02                    | 40 minutes      |
| 4        | Practice Task 03                    | 40 minutes      |
| 5        | Conclusion and wrap-up              | 20 minutes      |

## 10.3 Introduction

The **GROUP BY** clause allows SQL to organize rows that share common values into logical groups. Aggregate functions can then be applied to each group to produce summaries such as the number of students in each department, average age by department, or minimum and maximum values within a category. This approach is widely used in analytical reports and dashboard queries.

The **HAVING** clause works with grouped results. While **WHERE** filters individual rows before grouping occurs, **HAVING** filters groups after aggregate values have been calculated. Understanding this difference is important when building accurate analytical queries.

## 10.4 Walkthrough Task

Use the `Students` table to generate grouped summaries. First count students by department, then calculate multiple aggregate values. Apply `HAVING` to retain only groups that satisfy an aggregate condition, and finally sort the grouped output.

Listing 10.1: GROUP BY and HAVING examples

```
SELECT Department, COUNT(*) AS TotalStudents
FROM Students
GROUP BY Department;

SELECT Department,
       AVG(Age) AS AvgAge,
       MIN(Age) AS MinAge,
       MAX(Age) AS MaxAge
FROM Students
GROUP BY Department;

SELECT Department, COUNT(*) AS TotalStudents
FROM Students
GROUP BY Department
HAVING COUNT(*) > 5;

SELECT Department, COUNT(*) AS TotalStudents
FROM Students
GROUP BY Department
ORDER BY TotalStudents DESC;
```

## 10.5 Practice Tasks

### 10.5.1 Practice Task 1: GROUP BY

Group students by department and show the total number of students in each group. Calculate the average age for each department and display the minimum and maximum enrollment year for every department.

### 10.5.2 Practice Task 2: HAVING Clause

Display departments containing more than three students, departments with an average student age above 20, and departments whose student count falls between five and ten.

### 10.5.3 Practice Task 3: Advanced Grouping

Group students by enrollment year and count the number of students in each year. Display only years with more than two students, and then group students by department while retaining only departments where the minimum age is below 18.

## 10.6 Unseen Tasks

1. Create a grouped report showing the total number of students and average CGPA in each department, displaying only departments whose average CGPA is 3.0 or higher.
2. Group students by enrollment year and department, count the students in each combination, and sort the result first by year and then by student count in descending order.
3. Produce a report showing departments whose oldest student is above 24 years of age and whose total number of students is at least four.

## 10.7 Learning Outcomes

Students will be able to perform grouped aggregations, apply **HAVING** to filter aggregate results, distinguish between row-level and group-level filtering, and generate structured analytical reports using SQL.

## 10.8 Conclusion

The lab introduced grouped analysis using **GROUP BY** and **HAVING**. Students learned how to summarize data by category, apply aggregate calculations, filter groups using calculated conditions, and sort analytical output for reporting purposes.

# SQL JOINs: INNER JOIN and SELF JOIN

## Lab Objectives

1. Understand the purpose and structure of SQL JOIN operations.
2. Perform INNER JOIN operations between related tables.
3. Apply SELF JOIN to compare rows within the same table.
4. Interpret JOIN-based output in MySQL Workbench.
5. Strengthen relational analysis using primary-key and foreign-key relationships.

## 11.1 Reference Material

- W3Schools SQL JOIN Tutorial: [https://www.w3schools.com/sql/sql\\_join.asp](https://www.w3schools.com/sql/sql_join.asp)
- MySQL Documentation: <https://dev.mysql.com/doc/>
- TutorialsPoint SQL Self Join: <https://www.tutorialspoint.com/sql/sql-self-joins.html>

## 11.2 Activity Timeline

| Task No. | Activity                            | Time Allocation |
|----------|-------------------------------------|-----------------|
| 1        | Introduction and guided walkthrough | 20 minutes      |
| 2        | Practice Task 01                    | 40 minutes      |
| 3        | Practice Task 02                    | 40 minutes      |
| 4        | Practice Task 03                    | 40 minutes      |
| 5        | Conclusion and wrap-up              | 20 minutes      |

## 11.3 Introduction

Relational databases often distribute information across multiple tables to reduce duplication and improve data integrity. SQL JOIN operations reconnect those related tables at query time. An INNER JOIN returns rows where the join condition matches in both tables, commonly through a primary-key and foreign-key relationship. A SELF JOIN uses two aliases for the same table so that rows can be compared with other rows in that table.

In this lab, the `Students` table stores a `DepartmentID` foreign key, while the `Departments` table

stores the corresponding department name. This design allows department information to be stored once and referenced by many student records.

## 11.4 Walkthrough Task

Create or use the following related structures: `Students(StudentID, Name, Age, DepartmentID, EnrollmentDate)` and `Departments(DepartmentID, DepartmentName)`. Insert sample rows and execute joins that combine student and department information.

Listing 11.1: INNER JOIN between Students and Departments

```
SELECT Students.StudentID,  
       Students.Name,  
       Departments.DepartmentName  
FROM Students  
INNER JOIN Departments  
    ON Students.DepartmentID = Departments.DepartmentID;  
  
SELECT Students.Name,  
       Students.Age,  
       Departments.DepartmentName  
FROM Students  
INNER JOIN Departments  
    ON Students.DepartmentID = Departments.DepartmentID  
WHERE Students.Age > 20;
```

Listing 11.2: SELF JOIN example

```
SELECT A.Name AS StudentA,  
       B.Name AS StudentB,  
       A.EnrollmentDate  
FROM Students A  
INNER JOIN Students B  
    ON A.EnrollmentDate = B.EnrollmentDate  
    AND A.StudentID <> B.StudentID;
```

## 11.5 Practice Tasks

### 11.5.1 Practice Task 1: Basic INNER JOIN

Display student names with department names, retrieve students enrolled after 2022-01-01, sort joined results by department name, retrieve all Computer Science students, and display enrollment dates together with department names.

### 11.5.2 Practice Task 2: Moderate INNER JOIN

Count students in each department using `GROUP BY` with a join, show students older than 21 with department names, retrieve students whose department begins with the letter C, order joined output by age in descending order, and retrieve students whose department name ends with “Engineering”.

### 11.5.3 Practice Task 3: SELF JOIN

Display pairs of students with the same age, pairs sharing the same department, pairs enrolled in the same year, pairs whose names begin with the same letter, and pairs whose age difference is less than two years.

## 11.6 Unseen Tasks

1. Create an INNER JOIN query that shows each department name together with the number of students assigned to it, and sort the result from the largest department to the smallest.
2. Write a SELF JOIN that finds pairs of different students who belong to the same department and have an age difference of no more than one year. Ensure each pair appears only once.
3. Produce a joined report containing student name, department name, age, and enrollment year for students enrolled from 2022 onward, ordered by department and then by student name.

## 11.7 Learning Outcomes

Students will be able to combine related tables through INNER JOIN, compare records within one table using SELF JOIN, interpret key-based relationships, and construct multi-table reports for relational analysis.

## 11.8 Conclusion

SQL JOINS make normalized relational data useful by combining related information when it is needed. This lab demonstrated matched-row retrieval through INNER JOIN and row-to-row comparison through SELF JOIN, establishing essential skills for advanced query design and reporting.

---

# Introduction to MongoDB: Installation and CRUD Operations

---

## Lab Objectives

1. Understand the basic architecture and terminology of MongoDB.
2. Install MongoDB Community Server and MongoDB Compass.
3. Create databases, collections, and BSON documents.
4. Perform Create, Read, Update, and Delete operations using MongoDB Compass.
5. Compare fundamental characteristics of MongoDB and MySQL.

## 13.1 Reference Material

- MongoDB Community Server: <https://www.mongodb.com/try/download/community>
- MongoDB Compass: <https://www.mongodb.com/try/download/compass>
- MongoDB Documentation: <https://www.mongodb.com/docs/>

## 13.2 Software Required

- MongoDB Community Server
- MongoDB Compass
- Windows, Linux, or macOS
- Minimum recommended memory: 4 GB RAM

## 13.3 Introduction

MongoDB is a NoSQL, document-oriented database system. Unlike MySQL, which stores data in tables composed of rows and columns, MongoDB stores records as BSON documents inside collections. Documents can contain flexible sets of fields, making MongoDB useful when application data structures evolve frequently or when nested and semi-structured data must be stored naturally.

A MongoDB **database** contains collections, a **collection** contains documents, and a **document** represents an individual record. MongoDB Compass provides a graphical interface for creating and examining these objects, running filters, and performing CRUD operations without relying entirely on command-line tools.

| Feature         | MongoDB                                  | MySQL                         |
|-----------------|------------------------------------------|-------------------------------|
| Database model  | NoSQL, document-oriented                 | Relational SQL                |
| Storage         | BSON documents                           | Rows and columns              |
| Schema          | Flexible                                 | Fixed table schema            |
| Typical scaling | Horizontal scaling                       | Often vertical scaling        |
| Common use      | Rapidly changing or semi-structured data | Structured transactional data |

### 13.4 Walkthrough Task

Install MongoDB Community Server using the installer appropriate for the operating system. During installation, keep the standard service and port configuration unless the laboratory environment requires otherwise. Install MongoDB Compass and connect to the local server using `mongodb://localhost:27017`.

Create a database named `spme_db` with a collection named `students`. Insert student documents, retrieve records, apply a filtered read operation, update one document, and delete a selected document.

Listing 13.1: Sample MongoDB student document

```
{
  "student_id": 501,
  "name": "Ahmed Raza",
  "department": "Software Engineering",
  "semester": 5,
  "cgpa": 3.3
}
```

Listing 13.2: MongoDB Compass filters and update document

```
{ "cgpa": { "$gt": 3.0 } }

{ "student_id": 501 }

{ "$set": { "semester": 6 } }
```

### 13.5 Practice Tasks

#### 13.5.1 Practice Task 1: Build a Practice Database

Create a database named `practice_db` and a collection named `students`. Insert five student documents containing suitable fields such as student ID, name, department, semester, and CGPA.

#### 13.5.2 Practice Task 2: Retrieve and Filter Documents

Retrieve all students belonging to the Software Engineering department and then retrieve students whose CGPA is greater than 3.2. Verify that the filter returns only the expected documents.

### 13.5.3 Practice Task 3: Update and Delete Documents

Update the CGPA of one selected student and then delete one student document using `student_id`. Confirm both operations by retrieving the collection after the changes.

## 13.6 Unseen Tasks

1. Create a new collection named `courses` and insert at least five documents containing course code, course title, credit hours, and department. Retrieve only courses with more than three credit hours.
2. In the `students` collection, update the semester of all Software Engineering students by one level using an appropriate update operator, then verify the modified records.
3. Delete all student documents whose semester value is below 2 and whose CGPA is below 2.5, then display the remaining documents sorted by CGPA in descending order using Compass features.

## 13.7 Learning Outcomes

Students will be able to install and connect to MongoDB, explain core document-database terminology, create databases and collections, manipulate BSON documents through CRUD operations, and recognize key differences between document-oriented and relational database systems.

## 13.8 Conclusion

This lab introduced MongoDB and MongoDB Compass as a document-oriented alternative to relational database systems. Through installation, database creation, document insertion, filtering, updating, and deletion, students gained practical experience with MongoDB's flexible data model and essential CRUD workflow.

---

# MongoDB Advanced: Update, Delete and Aggregation Operations

---

## Lab Objectives

1. Perform multi-document update operations using MongoDB Compass.
2. Delete documents using condition-based filters.
3. Apply update operators such as `$set` and `$inc`.
4. Build aggregation pipelines for grouped counts and statistical calculations.
5. Interpret aggregation results for analytical reporting.

## 14.1 Reference Material

- MongoDB Documentation: <https://www.mongodb.com/docs/>
- MongoDB Compass Documentation: <https://www.mongodb.com/docs/compass/>

## 14.2 Software Required

- MongoDB Community Server
- MongoDB Compass
- Windows, Linux, or macOS

## 14.3 Introduction

MongoDB supports advanced data manipulation through update and delete operations that can affect one or many documents. Its Aggregation Framework provides a pipeline-based approach for transforming, grouping, and summarizing data. Conceptually, aggregation can perform tasks similar to SQL `GROUP BY`, aggregate functions, and filtering, while operating on MongoDB documents.

The aggregation pipeline processes documents through one or more stages. Each stage performs an operation such as filtering, grouping, projecting, sorting, or calculating values, and passes its output to the next stage. This structure allows complex analytical operations to be expressed in a clear sequence.

## 14.4 Walkthrough Task

Connect to MongoDB Compass through `mongodb://localhost:27017`, select the `spme_db` database, and open the `students` collection. Perform a multi-document update that changes semester 4 to

semester 5, delete students with CGPA below 2.5, and then use the Aggregations tab to count students by department and calculate the average CGPA.

Listing 14.1: Update and delete filters

```
Filter:
{ "semester": 4 }

Update:
{ "$set": { "semester": 5 } }

Delete Filter:
{ "cgpa": { "$lt": 2.5 } }
```

Listing 14.2: Aggregation pipeline for department-wise count

```
[
  {
    "$group": {
      "_id": "$department",
      "totalStudents": { "$sum": 1 }
    }
  }
]
```

Listing 14.3: Aggregation pipeline for average CGPA

```
[
  {
    "$group": {
      "_id": null,
      "averageCGPA": { "$avg": "$cgpa" }
    }
  }
]
```

## 14.5 Practice Tasks

### 14.5.1 Practice Task 1: Update Operations

Using a **students** collection, update all students in semester 3 to semester 4, increase the CGPA of all students by 0.2 using **\$inc**, and replace the department value “IT” with “Information Technology” for all matching documents.

### 14.5.2 Practice Task 2: Delete Operations

Delete all students whose CGPA is below 2.0, then delete one specific student using **student\_id**. Confirm the affected document count and verify the resulting collection after each operation.

### 14.5.3 Practice Task 3: Aggregation Operations

Use aggregation pipelines to count all student records, count students by department, calculate average CGPA, identify the maximum CGPA, and identify the minimum CGPA.

## 14.6 Unseen Tasks

1. Build an aggregation pipeline that groups students by semester and returns the total number of students and average CGPA for each semester, sorted from the highest average CGPA to the lowest.
2. Increase the CGPA of students in the Software Engineering department by 0.1 only when their current CGPA is below 3.5, then verify how many documents were modified.
3. Create an aggregation pipeline that first selects students with CGPA of at least 3.0, groups them by department, and returns the highest and lowest CGPA for each department.

## 14.7 Learning Outcomes

Students will be able to perform conditional multi-document updates and deletions, use common MongoDB update operators, construct aggregation pipelines, group documents for analysis, and calculate statistical summaries from collection data.

## 14.8 Conclusion

This lab extended basic MongoDB CRUD skills into multi-document manipulation and analytical processing. Students practiced conditional updates and deletions and used aggregation pipelines to group and summarize documents. These operations provide a foundation for data analysis and backend application development with MongoDB.